31901-2004 การพัฒนาซอฟต์แวร์ด้วยเทคโนโลยี Front-End

สัปดาห์ที่ 1–2 · ทฤษฎี 2 ชม. · ปฏิบัติ 6 ชม.

หน่วยที่ 1

ระบบควบคุมเวอร์ชัน (Git) และ GitHub

การจัดเก็บงานและการทำงานร่วมกันเป็นทีม ผ่านคำสั่ง Git และ Visual Studio 2026

ปวส.1 สาขาเทคโนโลยีสารสนเทศ · ภาคเรียนที่ 2/2569 · วิทยาลัยเทคโนโลยีภาคตะวันออก (อ.เทค)

จุดประสงค์เชิงพฤติกรรม

เมื่อจบหน่วยนี้ นักศึกษาสามารถ...

1 อธิบายหลักการทำงานของ Git (Working Directory, Staging Area, Repository)

2 ใช้คำสั่ง Git จัดเก็บงาน และติดตามการเปลี่ยนแปลง

3 ใช้ GitHub ส่งงาน แตก branch สร้าง Pull Request และรวมงานกับเพื่อนในทีม

4 แก้ไขความขัดแย้งของโค้ด (merge conflict)

แผนการเรียน

2 สัปดาห์ · 8 ชั่วโมง · 2 ใบงาน

สัปดาห์ที่ 1 · ท 1 + ป 3 ชม.

Git ในเครื่อง

- แนวคิด Git และการทำงาน 1 รอบ
- Git: init add commit log
- .gitignore, diff, restore

ใบงานที่ 1.1 RestaurantApp (รายบุคคล)

สัปดาห์ที่ 2 · ท 1 + ป 3 ชม.

ทำงานเป็นทีมบน GitHub

- remote, push, pull, clone
- branch, Pull Request, Code Review
- แก้ merge conflict

ใบงานที่ 1.2 RestaurantApp-Team (กลุ่ม 3 คน)

คำถามนำเข้าสู่บทเรียน

`งานกลุ่ม.zip` `งานกลุ่ม_final.zip` `งานกลุ่ม_final_v2.zip` `final_v2_แก้แล้ว_จริงๆ.zip`

ไฟล์ไหนคือล่าสุด? ใครแก้อะไร? ถ้าแก้พัง จะย้อนกลับยังไง?

ระบบควบคุมเวอร์ชัน (Git) ตอบทั้ง 3 คำถามนี้ โดยไม่ต้องส่งไฟล์ไปมา

1. ระบบควบคุมเวอร์ชัน (Version Control System)

บันทึกทุกการเปลี่ยนแปลง ไว้เป็นลำดับ

ย้อนกลับได้

กู้คืนงานเวอร์ชันก่อนหน้า

รู้ว่าใครแก้อะไร

ทุก commit มีชื่อ เวลา และข้อความ

ทำงานพร้อมกันได้

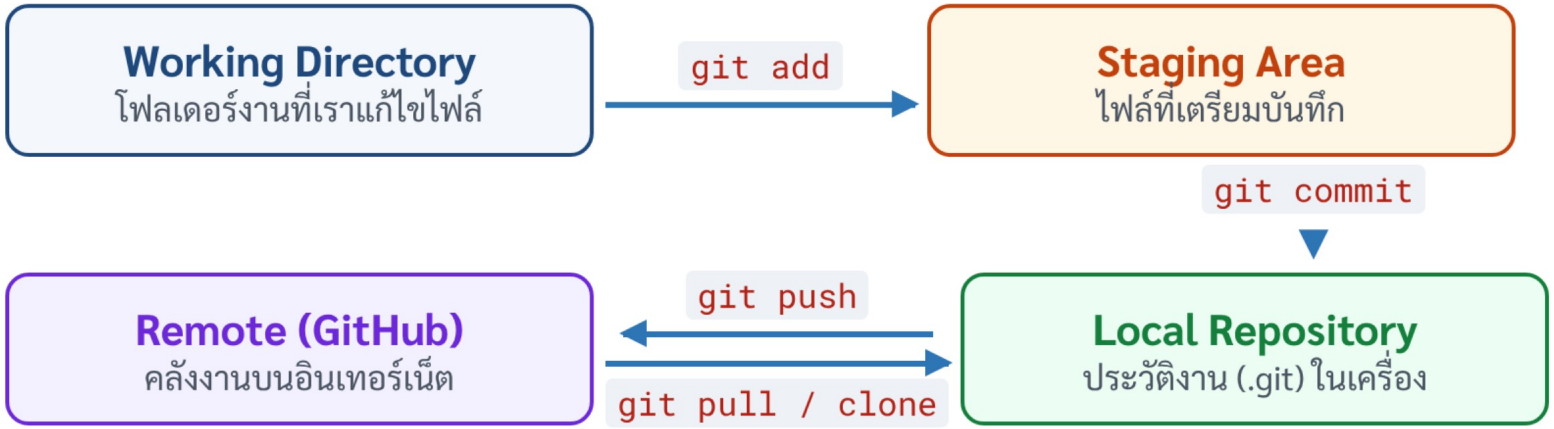
หลายคน โปรเจกต์เดียว ไม่ต้องส่งไฟล์

Git = โปรแกรมควบคุมเวอร์ชันแบบกระจาย ทำงานในเครื่อง

GitHub = บริการออนไลน์ เก็บ Repository ของ Git และทำงานร่วมกัน

1. พื้นที่ทำงานของ Git

งานเดินทางผ่าน 4 พื้นที่



Working Directory
โฟลเดอร์ที่กำลังแก้ไฟล์

Staging Area
ไฟล์ที่เลือกไว้รอบันทึก (git add)

Local Repository
ประวัติใน .git (git commit)

Remote
สำเนาบน GitHub (push / pull)

1. ศัพท์พื้นฐาน

ก่อนเขียนคำสั่ง ต้องรู้ว่าแต่ละคำคืออะไร

Repository

โฟลเดอร์โปรเจกต์ที่ Git เก็บประวัติทุก commit ไว้

เหมือน: แฟ้มงานพร้อมสมุดบันทึกทุกการแก้

commit

บันทึกงาน 1 ครั้ง พร้อมข้อความบอกว่าทำอะไร

เหมือน: กดเซฟเกม ย้อนกลับมาจุดนี้ได้

Remote / origin

Repository สำเนาบน GitHub · origin คือชื่อเล่นของที่อยู่นั้น

เหมือน: ตู้เก็บเอกสารกลางออนไลน์

push

ส่ง commit จากเครื่องเราขึ้น GitHub

เหมือน: อัปโหลดงานขึ้นตู้กลาง

pull

ดึง commit ใหม่จาก GitHub มารวมกับงานในเครื่อง

เหมือน: ดาวน์โหลดงานล่าสุดของเพื่อนมาอัปเดต

clone

คัดลอก Repository ทั้งก้อนจาก GitHub ลงเครื่อง ทำครั้งแรกครั้งเดียว

เหมือน: ถ่ายเอกสารทั้งแฟ้มมาเริ่มทำงาน

2. สมัครบัญชี GitHub

บัญชีเดียว ใช้ส่งงานตลอดเทอม

1. เข้า **github.com/signup** (หรือ github.com ▸ Sign up)
2. กรอก **Email** · ตั้ง **Password** · ตั้ง **Username**
3. ทำแบบทดสอบยืนยันว่าไม่ใช่บอท ▸ กรอก**รหัสยืนยัน**ที่ส่งไปทางอีเมล
4. เลือกแผน **Free** (คำถามแนะนำตัวกดข้ามได้)
5. รูปโปรไฟล์ ▸ Settings ▸ **Public profile** ใส่ชื่อจริงและรูป

Username
กลายเป็นที่อยู่งาน **github.com/<username>** ตั้งให้สุภาพ จำง่าย ใช้ได้ยาวถึงตอนสมัครงาน (อังกฤษ ตัวเลข ขีดกลาง) เช่น somchai-jd แล้วแจ้งครู

Email
ใช้อีเมลที่เปิดอ่านได้จริง เพราะรหัสยืนยันและคำเชิญเข้าทีมส่งมาทางอีเมล

2. ติดตั้งและตั้งค่า Git ครั้งแรก

บอก Git ว่าเราเป็นใคร และไม่ต้องเก็บอะไร

เปิด Terminal (View ▸ Terminal) แล้วพิมพ์ — ทำครั้งเดียวต่อเครื่อง

```
git --version
git config --global user.name "Somchai Jaidee"
git config --global user.email "somchai@e-tech.ac.th"
git config --global init.defaultBranch main
```

ชื่อและอีเมลนี้จะแสดงในทุก commit · **ชื่อ** = ชื่อจริงที่ครูอ่านแล้วรู้ว่าเป็นใคร · **อีเมล** = เดียวกับบัญชี GitHub · ตัวอย่างข้างบนเป็นชื่อสมมติ ให้ใส่ของตัวเอง

ถ้าขึ้น **'git' is not recognized** ให้เปิด Terminal ใหม่จาก **View ▸ Terminal** ใน VS (หรือติดตั้ง Git for Windows)

.gitignore = ไฟล์ที่บอก Git ว่าไม่ต้องเก็บอะไร

① **สร้างไฟล์** (พิมพ์ในโฟลเดอร์โปรเจกต์)

```
Set-Content .gitignore ".vs/","bin/","obj/"
```

② **ตรวจว่ามีเนื้อหา**

```
type .gitignore
# ผลที่ต้องเห็น 3 บรรทัด
.vs/
bin/
obj/
```

ต้องทำ**ก่อน git add ครั้งแรก** · ถ้า type ไม่แสดงอะไร แปลว่าไฟล์ว่าง ให้สั่ง ① ใหม่

2. คำสั่งตั้งต้น

ทำครั้งเดียว: แต่ละคำสั่งคืออะไร

คำสั่ง	ทำอะไร	ทำเมื่อไหร่
git --version	เช็กว่าเครื่องมี Git และใช้ได้	ครั้งแรกต่อเครื่อง
git config --global user.name "ชื่อ"	ตั้งชื่อที่จะขึ้นในทุก commit	ครั้งเดียวต่อเครื่อง
git config --global user.email "อีเมล"	ตั้งอีเมล (ให้ตรงกับบัญชี GitHub)	ครั้งเดียวต่อเครื่อง
git init -b main	ทำให้โฟลเดอร์นี้เป็น Repository (สร้างโฟลเดอร์ซ่อน .git) โดยตั้ง branch หลักชื่อ main	ครั้งเดียวต่อโปรเจกต์
.gitignore (ไฟล์)	รายชื่อสิ่งที่ไม่ต้องเก็บ: .vs/ bin/ obj/ ต้องสร้างก่อน git add ครั้งแรก	ก่อน add ครั้งแรก
git remote add origin <URL>	ตั้งชื่อเล่น origin ให้ที่อยู่ Repository บน GitHub	ครั้งเดียวต่อโปรเจกต์
git push -u origin main	ส่งขึ้น GitHub ครั้งแรกและจำเส้นทาง (-u) ต่อไปพิมพ์แค่ git push	ครั้งแรกครั้งเดียว

3. คำสั่ง Git พื้นฐาน

8 คำสั่งที่ใช้ทุกวัน

คำสั่ง	หน้าที่
git init	สร้าง Repository ใหม่ในโฟลเดอร์ปัจจุบัน
git status	ดูสถานะไฟล์: แก้แล้ว / อยู่ใน Staging / ยังไม่ติดตาม
git add <ไฟล์> · git add .	นำไฟล์เข้าสู่ Staging Area
git commit -m "ข้อความ"	บันทึกสิ่งที่อยู่ใน Staging เป็น commit ใหม่
git log --oneline	ดูประวัติ commit แบบย่อ
git diff	ดูความแตกต่างของไฟล์ที่ยังไม่ได้ add
git restore <ไฟล์>	ยกเลิกการแก้ไข กลับไปเหมือน commit ล่าสุด
git revert <hash>	สร้าง commit ใหม่ที่ย้อนผล commit ที่ระบุ (ปลอดภัยกับงานทีม)

3. ตัวอย่างการทำงานจริง

จากโฟลเดอร์เปล่า สู่ commit ที่ 2

```
cd RestaurantApp
git init -b main
Set-Content .gitignore ".vs/","bin/","obj/"
type .gitignore      # ตรวจว่ามีเนื้อหา
git add .
git commit -m "สร้างโปรเจกต์ RestaurantApp"

# ... แก้ไฟล์ Program.cs แล้วกด Ctrl+S ...
git status
git add Program.cs
git commit -m "เพิ่มเมนูแสดงรายการร้าน"
git log --oneline
```

git status สีแดง
แก้แล้ว ยังไม่ add

git status สีเขียว
อยู่ใน Staging พร้อม commit

nothing to commit
ทุกอย่างบันทึกแล้ว

3. ประวัติ commit

commit ต่อกันเป็นสาย · HEAD ชี้ commit ล่าสุด



แต่ละ commit คือ "ภาพถ่าย" ของงานทั้งโปรเจกต์ ณ เวลานั้น มีรหัส (hash) และข้อความอธิบาย

ข้อความ commit ที่ดี

ดี: บอกว่าทำอะไร
"เพิ่มเมนูค้นหาร้านตามชื่อ"

ไม่ดี: ไม่บอกอะไร
"แก้ไข" · "update" · "asdf"

1 commit = 1 เรื่อง
ทำเสร็จ และ compile ผ่าน

commit บ่อย ๆ
ย้อนกลับได้ละเอียดกว่า

4. หลักตั้งชื่อ Repository

คิดก่อนกดสร้าง: ชื่อนี้ต้องตรงกับอะไร

1

ตรงกับชื่อโปรเจกต์

ใช้ชื่อเดียวกับโปรเจกต์ใน VS จะได้รู้ว่าอันไหนคู่กัน

2

บอกว่าเป็นงานอะไร

คนอื่นอ่านชื่อแล้วเข้าใจ ไม่ใช้ชื่ออย่าง test หรือ งาน1

3

อังกฤษ ไม่เว้นวรรค

ใช้ตัวอักษร ตัวเลข และ - เพราะชื่อกลายเป็นส่วนของ URL

4

งานทีมบอกว่าเป็นของทีม

เติม -Team ท้ายชื่อ เพื่อแยกจากงานรายบุคคล

นำหลักมาใช้: ใบงาน 1.1 → **RestaurantApp** (ชื่อเดียวกับโปรเจกต์) · ใบงาน 1.2 → **RestaurantApp-Team**

ตัวเลือกตอนสร้าง: ไม่ติ๊ก README

ถ้ามีงานในเครื่องแล้ว อย่าติ๊ก README / .gitignore / License เพราะ GitHub จะสร้าง commit ที่เครื่องเราไม่มี ทำให้ push ครั้งแรกไม่ผ่าน

Private หรือ Public

Private = เฉพาะคนที่เราเชิญเห็น · **Public** = ใครก็เห็นได้ ให้เลือกตามที่ครูกำหนด

4. GitHub และ Remote Repository

ส่งงานจากเครื่องขึ้น GitHub

1. github.com ▸ ล็อกอิน ▸ **New repository**
2. ตั้งชื่อตามหลักในหน้าก่อน (ใบงานนี้ **RestaurantApp**) · **ไม่ติ๊ก** README (มีงานในเครื่องแล้ว)
3. คัดลอก URL ของ Repository
4. เชื่อมต่อแล้ว push ครั้งแรก (VS จะให้ล็อกอิน GitHub)

```
# origin = ชื่อเล่นของที่อยู่บน GitHub
git remote add origin https://github.com/
  <ชื่อผู้ใช้>/RestaurantApp.git
git push -u origin main
# -u จำว่า main ผูกกับ origin/main

# เครื่องอื่น/เพื่อนในทีม คัดลอกลงเครื่อง
git clone https://github.com/
  <ชื่อผู้ใช้>/RestaurantApp.git

# ดึงงานล่าสุดจาก GitHub
git pull
```

5. การทำงานระหว่างเครื่องกับ GitHub

งานเดินทางไปมาอย่างไร

1. **เรา** แก้งานแล้ว **commit** → บันทึกอยู่ในเครื่องเรา ยังไม่มีใครเห็น
2. **เรา push** → งานขึ้นไปอยู่บน GitHub
3. **เพื่อน clone** → ได้สำเนาทั้งโปรเจกต์ลงเครื่องเพื่อน
4. **เพื่อน** แก้งาน **commit** แล้ว **push** → GitHub มีงานใหม่ที่เครื่องเรายังไม่มี
5. **เรา pull** → งานใหม่ของเพื่อนมาอยู่ในเครื่องเรา

จำง่ายๆ
เริ่มงาน → pull ดึงงานล่าสุดมาก่อน
จบงาน → commit แล้ว push ส่งขึ้น

push ไม่ผ่าน (rejected)
แปลว่า GitHub มีงานใหม่ที่เรายังไม่มี ให้ pull ก่อน แล้ว push ใหม่

5. ภาพรวมการทำงานของ Git

ทำงาน 1 รอบ = 5 ขั้น

1

แก้ไฟล์

```
Ctrl + S
```

เขียนโค้ดใน VS แล้วกดบันทึก

→

2

ตรวจ

```
git status
git diff
```

ดูว่า ไฟล์ไหนเปลี่ยน และเปลี่ยนอะไร

→

3

เลือก

```
git add .
```

เลือกไฟล์ ที่จะบันทึกเข้า Staging

→

4

บันทึก

```
git commit -m "ข้อความ"
```

เก็บเป็น 1 commit พร้อมข้อความ บอกว่าทำอะไร

→

5

ส่งขึ้น

```
git push
```

ส่ง commit ขึ้น GitHub ให้เพื่อนเห็น

ทำครั้งเดียว (ครั้งแรกของโปรเจกต์)

git init → สร้าง .gitignore → git remote add origin

ทำซ้ำทุกรอบ (ขั้น 1–5)

แก้ → ตรวจ → เลือก → บันทึก → ส่งขึ้น · ทำบ่อยๆ ทีละเรื่อง

6. สรุปขั้นตอน

ทำ 1 ครั้ง ต้องทำอะไรบ้าง

ครั้งแรกของโปรเจกต์ (เตรียม)

1. เปิด Terminal ใน VS (View ▸ Terminal) แล้วตรวจ **git --version**
2. ตั้งชื่อและอีเมล **git config --global user.name** และ **user.email**
3. **git init -b main** แล้วสร้าง **.gitignore** ตรวจด้วย **type .gitignore**
4. สร้าง Repository บน GitHub แล้ว **git remote add origin <URL>**

ทุกครั้งที่แก้งาน (วนซ้ำ)

1. แก้ไฟล์ แล้วกด **Ctrl+S**
2. **git status** · **git diff** ดูว่าอะไรเปลี่ยน (แดง = ยังไม่ add)
3. **git add .** เลือกไฟล์เข้า Staging (เขียว = พร้อม commit)
4. **git commit -m "ทำอะไร"** บันทึกเป็น 1 commit
5. **git push** ส่งขึ้น GitHub (ครั้งแรกใช้ **git push -u origin main**)



ดูประวัติได้ทุกเมื่อด้วย **git log --oneline** · commit บ่อยๆ ทีละเรื่อง ข้อความบอกว่าทำอะไร

6. ปัญหาที่พบบ่อยตอนใช้ Git ในเครื่อง

อาการ → สาเหตุ → วิธีแก้

อาการ	สาเหตุ	วิธีแก้ไข
ขึ้นแดง "'git' is not recognized"	เทอร์มินัลถูกเปิดไว้ก่อนติดตั้ง Git หรือเครื่องยังไม่มี Git for Windows	เปิด Terminal ใหม่จาก View ▸ Terminal ใน VS · ถ้ายังไม่ได้ติดตั้ง Git for Windows แล้วเปิดใหม่
git add . ขึ้น Permission denied (.vs)	ยังไม่มี .gitignore กันโฟลเดอร์ .vs ที่ VS เปิดล็อกไว้	สร้าง .gitignore ที่มี .vs/ bin/ obj/ แล้วสั่ง git add . ใหม่ (ถ้ายังไม่ผ่าน ปิด Solution ใน VS ก่อน)
type .gitignore ไม่แสดงอะไร	ไฟล์ว่าง เพราะไม่ได้กด Ctrl+S ใน Notepad หรือบันทึกเป็น .gitignore.txt	เขียนไฟล์ให้มี .vs/ bin/ obj/ แล้วบันทึก · ตรวจชื่อไฟล์ด้วย dir -Force
commit แล้วขึ้น Author identity unknown	ยังไม่ได้ตั้ง user.name และ user.email	git config --global user.name และ user.email แล้ว commit ใหม่
อยากแก้ข้อความ commit ล่าสุด	พิมพ์ผิดหรือบอกไม่ชัด (และยังไม่ได้ push)	git commit --amend -m "ข้อความใหม่"

ใบงานที่ 1.1 · สัปดาห์ที่ 1 · รายบุคคล · ท 1 ป 3 ชม.

RestaurantApp: รายชื่อร้านอาหาร และเก็บงานด้วย Git

ใบงานบอกแค่ลำดับขั้นตอน · เขียนโค้ดและเลือกคำสั่ง Git เอง · ดูคำสั่งจากใบสรุปขั้นตอน (หัวข้อ 6)

1	สร้าง Console App (.NET Framework 4.8) ชื่อ RestaurantApp
2	ตั้งค่าชื่อและอีเมลของ Git
3	ทำให้โฟลเดอร์เป็น Repository (branch main)
4	สร้าง .gitignore (.vs bin obj) แล้วตรวจเนื้อหา
5	ตรวจสถานะ แล้ว commit แรก
6	สร้างคลาส Restaurant → commit
7	สร้าง List ร้าน 1–3 → commit
8	แสดงรายชื่อร้านทั้งหมด → commit
9	เพิ่มร้าน 4–5 · ตรวจสิ่งที่เปลี่ยนก่อน → commit
10	แสดงจำนวนร้าน + คะแนนเฉลี่ย → commit
11	ทำโค้ดพัง แล้วคืนไฟล์ด้วยคำสั่ง Git
12	ดูประวัติ commit (≥ 6)
13	สร้าง Repository RestaurantApp บน GitHub แล้วเชื่อม
14	ส่งงานขึ้น GitHub · ตรวจบนเว็บ
15	แก้เพิ่ม 1 อย่างตามความคิดตนเอง → commit → ส่งขึ้นอีกครั้ง
16	ส่งลิงก์ Repository + ภาพประวัติ commit

7. Clone งานของทีมด้วย Visual Studio

รับคำเชิญ แล้วดึงงานลงเครื่อง

1. เปิดอีเมลคำเชิญ หรือ github.com/notifications ▸ **Accept invitation**
2. VS หน้า Start ▸ **Clone a repository**
3. **Browse a repository** ▸ **GitHub** ▸ เลือก Repository ของทีม
4. เลือก Path ที่เก็บงาน ▸ **Clone**
5. Solution Explorer ▸ ดับเบิลคลิกไฟล์ **.sln**

```
# เทียบกับคำสั่ง
git clone https://github.com/
  <owner>/RestaurantApp.git
```

ยังไม่เห็น Repository
ยังไม่ได้ Accept invitation หรือ Sign in VS ด้วยบัญชีอื่น

เลือก Path
อย่า Clone ซ้อนในโฟลเดอร์ที่มีงานอยู่แล้ว หรือโฟลเดอร์ OneDrive

8. ศัพท์ของการทำงานเป็นทีม

ก่อนทำงานเป็นทีม รู้จัก 4 คำนี้

branch

สายงานแยกออกจาก main แก้ไขได้โดยไม่กระทบงานหลัก

เหมือน: ถ่ายสำเนาไปแก้ร่างก่อน

merge

รวมงานจาก branch กลับเข้า main

เหมือน: เอาร่างที่แก้เสร็จไปรวมกับต้นฉบับ

Pull Request (PR)

คำขอบน GitHub ให้เพื่อนตรวจงาน ก่อนรวมเข้า main

เหมือน: ส่งงานให้เพื่อนตรวจก่อนรวม

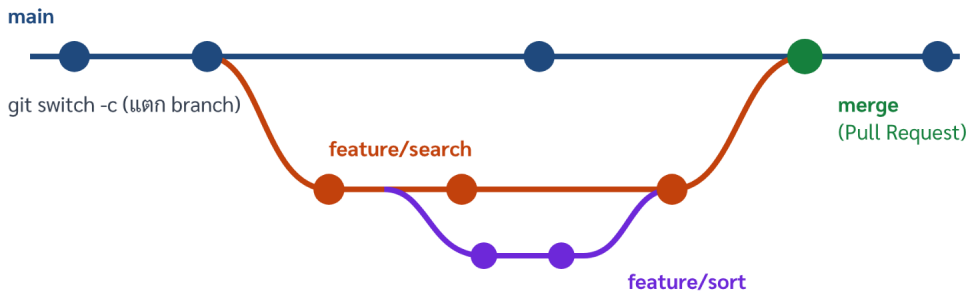
merge conflict

2 คนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกอันไหน ต้องให้คนเลือก

เหมือน: สองคนแก้ประโยคเดียวกันคนละแบบ

8. Branch และการรวมงาน (Merge)

แตกสายไปทำฟีเจอร์ แล้วรวมกลับ main



แต่ละคนแตก branch ทำงานของตนเองโดยไม่กระทบ main แล้วรวมงานกลับด้วย merge

```
git switch -c feature/sort-rating
# ... แก้ไขโค้ด ...
git add .
git commit -m "เพิ่มเมนูเรียงคะแนน"
git push -u origin feature/sort-rating

git switch main
git merge feature/sort-rating
git branch -d feature/sort-rating
```

switch -c สร้างและย้ายไป branch ใหม่

merge รวม branch เข้า branch ปัจจุบัน

branch -d ลบ branch ที่รวมแล้ว

9. หลักตั้งชื่อ branch และเขียน Pull Request

ชื่อที่ดี อ่านแล้วรู้ว่าทำอะไร

ชื่อ branch

รูปแบบ: ประเภท/ชื่องาน

ประเภท: feature = เพิ่มความสามารถ · fix = แก้ข้อผิดพลาด

ชื่องาน: อังกฤษตัวเล็ก คั่นด้วย - ไม่เว้นวรรค

1 branch = 1 งาน ทำเสร็จแล้วรวมและลบทิ้ง

```
feature/sort-rating
fix/search-empty
```

Pull Request

ชื่อ PR: เหมือนข้อความ commit บอกว่าทำอะไร เช่น "เพิ่มเมนูเรียงตามคะแนน"

รายละเอียด: เขียนให้เพื่อนตรวจได้ 3 เรื่อง

① ทำอะไรไป
② ทดสอบอย่างไร
③ อยากให้เพื่อนดูตรงไหน

ใบงาน 1.2 กำหนดชื่อ branch ให้แล้ว (feature/add-restaurant · feature/good-rating · feature/summary) ลองดูว่าตรงกับหลักข้อไหน

9. การทำงานเป็นทีมด้วย Pull Request

ไม่ merge เอง · ให้เพื่อนตรวจโค้ดก่อนรวม



Pull Request (PR) = คำขอรวม branch เข้า main ที่เปิดให้ทีมอ่าน แสดงความเห็น และอนุมัติก่อน

9. ขั้นตอนบน GitHub

เชิญ → เปิด PR → ตรวจ → รวม → อัปเดต

ขั้นตอน	วิธีทำ
เชิญสมาชิก	Settings ▸ Collaborators ▸ Add people (เพื่อนกดยอมรับทางอีเมล)
เปิด Pull Request	หลัง push branch ▸ Compare & pull request ▸ ตั้งชื่อและเขียนสรุปตามหลักตั้งชื่อ PR ▸ Create pull request
ตรวจโค้ด	แท็บ Files changed ▸ แสดงความเห็นรายบรรทัด ▸ Review changes ▸ Approve
รวมงาน	Merge pull request ▸ Confirm merge ▸ Delete branch
อัปเดตเครื่องตนเอง	git switch main · git pull

10. การแก้ไขความขัดแย้ง (Merge Conflict)

2 branch แก้บรรทัดเดียวกัน · Git ให้เราเลือก

ไฟล์ที่เกิด conflict

```
<<<<<<< HEAD                    // โค้ดใน branch ปัจจุบัน
Console.WriteLine("เมนูหลัก");
=======
Console.WriteLine("=== Main Menu ===");     // โค้ดที่กำลังรวมเข้ามา
>>>>>>> feature/menu
```

▼ แก้ไขแล้ว

```
Console.WriteLine("=== เมนูหลัก ===");
```

<<<<<<< HEAD
โค้ดฝั่งของเรา (Current)

=======
เส้นแบ่ง 2 ฝั่ง

>>>>>>> main
โค้ดที่เข้ามา (Incoming)

10. ขั้นตอนแก้ conflict

เปิด → เลือก → ลบเครื่องหมาย → commit

1

เปิดไฟล์ที่ขัดแย้ง VS แสดงใน **Merge Editor** เลือก Incoming / Current ได้

2

อ่านว่าแต่ละฝั่งตั้งใจแก้อะไร (ไม่แน่ใจให้ถามเพื่อน) แล้วเลือกฝั่งที่ถูก หรือรวมทั้งสองส่วน

3

ลบบรรทัด <<<<<<< ======= >>>>>>> ให้หมด แล้ว build ให้ผ่าน

4

git add <ไฟล์> แล้ว git commit บันทึกผลการแก้

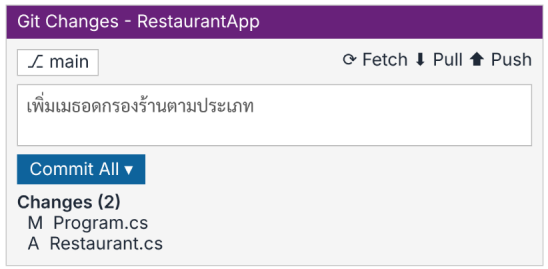
⚠ **ข้อควรระวัง**

ก่อนเริ่มงานทุกครั้ง **git pull** เพื่อลดโอกาสเกิด conflict

ห้ามใช้ **git push --force** กับ branch ที่ทีมใช้ร่วมกัน เพราะจะลบงานของเพื่อน

11. ใช้ Git ใน Visual Studio 2026

ทุกคำสั่ง กดผ่านหน้าจอได้



ภาพจำลองหน้าต่าง Git Changes ใน Visual Studio (เมนู View ▸ Git Changes)

งาน	เมนูใน VS 2026
สร้าง Repository	Git ▸ Create Git Repository
commit	View ▸ Git Changes ▸ พิมพ์ข้อความ ▸ Commit All
push / pull	ปุ่มลูกศรขึ้น / ลง ใน Git Changes
สร้าง / สลับ branch	Git ▸ New Branch หรือคลิกชื่อ branch ที่แถบล่าง
ดูประวัติ	Git ▸ View Branch History
clone	หน้า Start ▸ Clone a repository

11. ก่อนเลิกใช้เครื่องในห้องแล็บ

ออกจากบัญชีทุกครั้ง กันคนอื่นส่งงานในชื่อเรา

1. VS ▸ File ▸ Account Settings ▸ บัญชี GitHub ▸ **Remove** (ถ้าเคยเพิ่มไว้)
2. Windows ▸ **Credential Manager** ▸ Windows Credentials ▸ ลบ **git:https://github.com**
3. เบราว์เซอร์ ▸ github.com ▸ **Sign out**
4. Terminal: **git config --global --unset user.name** และ **user.email**

ถ้าลืมออกจากบัญชี
คนถัดไป push แล้วขึ้นชื่อเรา หรือ push ไม่ได้ (403) เพราะเครื่องจำบัญชีเก่า

ทำให้เป็นนิสัย
ปิดคาบทุกครั้ง: push งาน → ออกจากบัญชี → ปิดเครื่อง

11. ปัญหาที่พบบ่อยเมื่อเชื่อม GitHub

อาการ → สาเหตุ → วิธีแก้

อาการ	สาเหตุ	วิธีแก้ไข
push แล้วถามให้ Sign in ทุกครั้ง	ยังไม่เพิ่มบัญชีใน VS หรือกดยกเลิกหน้าต่าง Sign in	push ใหม่ แล้วกด Sign in with browser ในหน้าต่างที่เด้ง (หรือ Account Settings ▸ Add GitHub)
403 Permission denied	เครื่องจำบัญชีคนก่อน หรือยังไม่ได้รับเชิญเข้า Repository	ลบใน Credential Manager · Accept invitation
rejected (fetch first)	บน GitHub มีงานใหม่ที่เครื่องยังไม่มี	git pull ก่อน แล้วค่อย push
commit ขึ้นชื่อคนอื่น	user.name / user.email ยังเป็นของคนก่อน	ตั้ง git config ใหม่ก่อน commit
bin / obj ขึ้นไปบน GitHub	ไม่มี .gitignore ตอน commit แรก	เพิ่ม .gitignore แล้ว git rm -r --cached bin obj
Clone แล้วไม่เห็นโปรเจกต์	เปิดผิดโฟลเดอร์ หรือ Repository ยังว่าง	เปิดไฟล์ .sln ในโฟลเดอร์ที่ Clone

ใบงานที่ 1.2 · สัปดาห์ที่ 2 · กลุ่ม 3 คน · ท 1 ป 3 ชม.

RestaurantApp-Team: Branch, Pull Request, Merge Conflict

หัวหน้าเตรียมโครง เมธอด แล้วส่งขึ้น GitHub → เชิญสมาชิก · ทุกคน clone → คนละ branch คนละความสามารถ → PR + Review + Merge → ฝึก conflict · README

คน	ชื่อ branch	ความสามารถที่ต้องเพิ่ม (เขียนโค้ดเอง)
A หัวหน้า	feature/add-restaurant	รับชื่อร้าน ประเภท คะแนนจากคีย์บอร์ด แล้วเพิ่มเข้า List
B	feature/good-rating	แสดงเฉพาะร้านที่คะแนนตั้งแต่ 4.5 ขึ้นไป
C	feature/summary	แสดงร้านคะแนนสูงสุด และร้านคะแนนต่ำสุด

⚠ ใบงานบอกแค่ลำดับขั้นตอน เขียนโค้ดและเลือกคำสั่งเอง · ดึงงานล่าสุดก่อนแก้ · ห้ามส่งตรงเข้า main · ห้าม push --force

ใบงานที่ 1.2 · ฝึก conflict

ทุกคนแก้หัวเมนู "=== Restaurant Menu ===" คนละแบบ

1. แต่ละคนแตก branch ใหม่ feature/title-<ชื่อ> แก้หัวเมนู แล้วเปิด PR
2. คนแรก merge ได้ · คนถัดไป GitHub แจ้ง conflict
3. ดึงงานล่าสุดจาก main มารวม แก้ไฟล์ แล้วส่งขึ้นซ้ำ
4. หัวหน้าเพิ่ม README.md (ชื่อสมาชิก หน้าที่ วิธีรัน) ผ่าน PR

ทำเอง: คิดคำสั่งเอง
ทบทวนวิธีจากหัวข้อ 10 (Merge Conflict) แล้วลงมือเอง ไม่มีคำสั่งให้ในใบงาน

เสร็จเมื่อ
ไม่มีเครื่องหมาย <<<<<<< ======= >>>>>>> เหลือในไฟล์ · build ผ่าน · PR รวมเข้า main ได้